

DCQO Trotterization Upgrades for NVIDIA LABS Challenge

Context

The NVIDIA LABS challenge uses **Digitized Counterdiabatic Quantum Optimization (DCQO)** as a *sampling primitive* to generate high-quality bitstrings that seed a classical Memetic Tabu Search (MTS). The goal of the quantum component is **not accurate Hamiltonian simulation**, but rather **low-depth, biased sampling** that improves downstream classical optimization.

The tutorial notebook (`01_quantum_enhanced_optimization_LABS.ipynb`) implements DCQO following Cadavid et al. using a **first-order (Lie-Trotter) product formula** with a time-dependent counterdiabatic angle schedule. This document describes three targeted upgrades to that trotterization which: - preserve the first-order / low-depth structure of DCQO, - introduce **no additional circuit depth**, and - improve discretization accuracy and sampling diversity.

Baseline Method in the Current Notebook

The notebook currently uses:

1. First-order Suzuki-Trotter (Lie-Trotter) product formula

$$e^{-i\theta \sum_j P_j} \approx \prod_j e^{-i\theta P_j}$$

where the Pauli strings P_j correspond to the G2/G4 terms in the DCQO generator.

2. Right-endpoint time discretization for the time-dependent CD angle:

$$t_k = k \Delta t$$

3. Fixed Pauli ordering

4. The same deterministic ordering of Pauli terms is used for every step and every run.

This is a valid and faithful implementation of DCQO, but it leaves avoidable discretization and product-formula error on the table.

Upgrade 1: Midpoint Time Discretization

What changes

Replace right-endpoint sampling

$$t_k = k \Delta t$$

with **midpoint sampling**

$$t_k = \left(k - \frac{1}{2}\right) \Delta t$$

when computing the CD rotation angle θ_k .

Why this was chosen

- For time-dependent generators, midpoint sampling improves local integration accuracy from **$\mathcal{O}(\Delta t)$** to **$\mathcal{O}(\Delta t^2)$** .
- This improvement requires **no additional quantum gates** and **no increase in circuit depth**.
- DCQO explicitly relies on discretizing a continuous schedule; improving that discretization directly improves sampling quality.

How it differs from the notebook

- The notebook evaluates $\theta(t)$ at the **end** of each interval.
- The upgrade evaluates $\theta(t)$ at the **center** of each interval.
- All other aspects of the circuit remain unchanged.

References

- Ikeda et al., *Improved Time-Dependent Product Formulas*, Quantum **7**, 1168 (2023)
- Childs et al., *Theory of Trotter Error with Commutator Scaling*, Phys. Rev. X **11**, 011020 (2021)

Upgrade 2: Forward / Reverse Pauli Ordering (Deterministic Averaging)

What changes

Alternate the ordering of Pauli-string exponentials between steps: - Even steps: forward ordering - Odd steps: reverse ordering

Why this was chosen

- Fixed ordering causes **coherent first-order product-formula error** to accumulate in the same direction.
- Alternating forward/reverse ordering cancels this error **in expectation**, without increasing depth.

- This technique is deterministic, kernel-friendly, and compatible with CUDA-Q constraints.

How it differs from the notebook

- The notebook always applies Pauli terms in the same fixed order.
- The upgrade introduces a simple parity-based reversal that averages ordering errors over the circuit.

References

- Childs, Ostrander, Su, *Faster Quantum Simulation by Randomization*, Quantum **3**, 182 (2019)
-

Upgrade 3: K-Permutation Averaged Ordering (Randomized Product Formula Lite)

What changes

Instead of a single Pauli ordering, precompute a small set of K permutations of the Pauli terms (e.g., $K = 8-32$). Different runs or batches use different permutations.

Why this was chosen

- Randomized product formulas have stronger theoretical error bounds than fixed-order formulas.
- For DCQO, this also **increases the diversity of sampled bitstrings**, which is beneficial for population-based classical optimizers like MTS.
- Implemented carefully, this adds **no additional gates** and only minimal classical overhead.

How it differs from the notebook

- The notebook uses exactly one Pauli ordering.
- The upgrade averages over multiple orderings across runs or shot batches.

References

- Childs, Ostrander, Su, *Faster Quantum Simulation by Randomization*, Quantum **3**, 182 (2019)
-

Why These Three Upgrades (and Not Higher-Order Suzuki-Trotter)

Although higher-order Suzuki-Trotter formulas (e.g., Strang splitting) reduce simulation error, they **increase circuit depth**. DCQO's advantage, as emphasized by Cadavid et al., is its effectiveness in the **low-depth impulse regime**. Increasing depth undermines that advantage.

The chosen upgrades: - preserve first-order DCQO structure, - improve discretization and ordering error, - add zero quantum depth, - and directly target sampling quality rather than simulation fidelity.

Summary

Baseline: - First-order Suzuki–Trotter - Right-endpoint time sampling - Fixed Pauli ordering

Upgraded DCQO Trotterization: 1. Midpoint time discretization 2. Forward/reverse Pauli ordering 3. K-permutation averaged ordering

Together, these changes tighten the DCQO digitization, improve low-depth sampling quality and diversity, and align with both the Cadavid methodology and the NVIDIA LABS evaluation criteria—without increasing circuit depth.

Primary DCQO Reference

- Cadavid et al., *Scaling Advantage with Quantum-Enhanced Memetic Tabu Search for LABS*, arXiv: 2511.04553